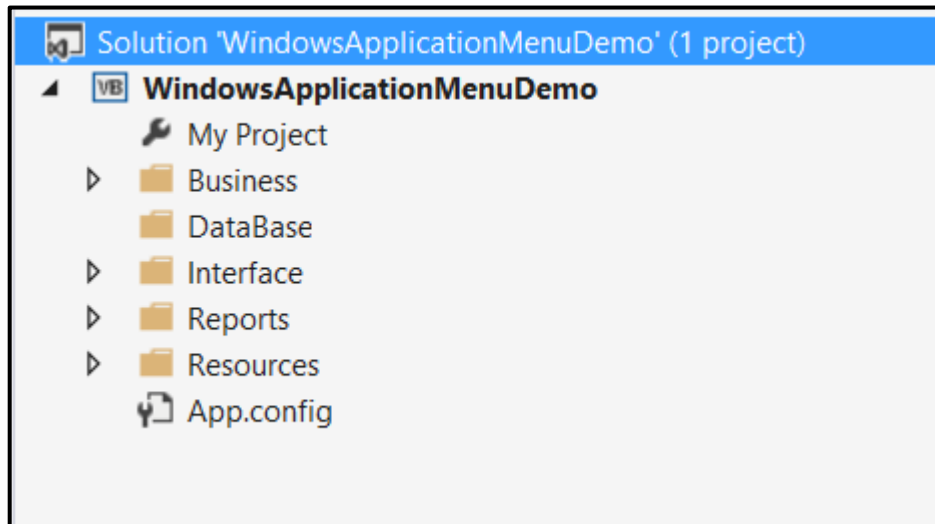


ISTN 3AS – MAJOR PROJECT PRACTICAL SESSION 1

MENU CONSTRUCTION

Arranging the Workspace

When developing a complex Windows based solution to a problem, it is always a good idea to arrange the Solution Explorer into folders. For the current application the following folder arrangement is suggested:



The purpose of the folders is to arrange the many forms that the application will contain into logical containers. The Business folder will contain all forms that pertain to the business processing part of the solution. The Interface folder will contain the Main Menu form and the Login form. The Database folder will contain all objects that enable access to the database. The resources folder will contain all external entities (mostly jpeg images) that are required for the application. The App.Config file is a text file (XML) that contains configuration data that is used by the compiler to control application settings such as a reference to the compiler version, the location of the external data source (usually a Sql Server address) and a reference to the plugin tool used to create reports for the application.

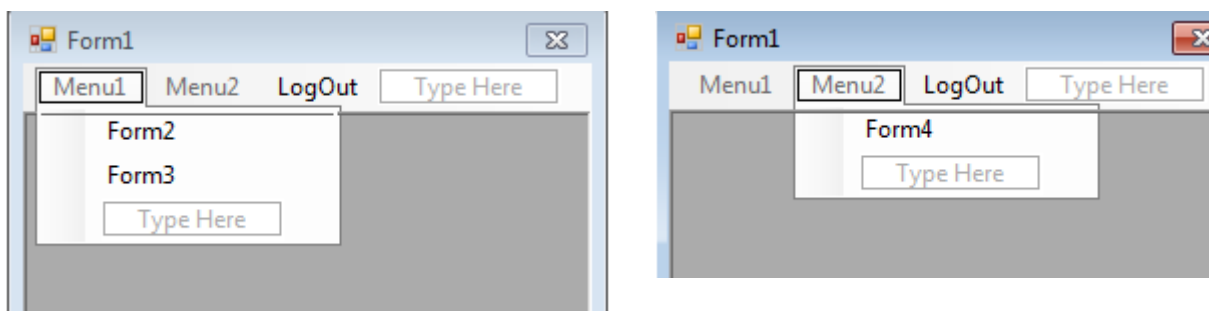
Prac 1 Part One: A Multi Document Interface (MDI) Menu System

This is a demo prac to understand the difference between MDIs and SDI (optional prac)

A MDI menu system is more professional because of the following features:

- Consists of an outer window (called the parent) that contains the menu system
- And an inner window that contains the content (called the child window)

- Much more flexible because different views can be shown to the user whilst keeping the menu display constant – i.e. by changing the child windows all the time, different types of content can be shown. The user is at liberty to switch between the different views. E.g. Ms-Word
1. Create main form and set it to be the Parent Form (i.e. Form1 becomes the main form – In MDI applications, you have a main form that will contain the menu system and all other forms (referred to as child forms “feed” off this menu system. The main form is referred to as the *MDIContainer*. In order to set this as the MDIContainer, go to the propertieslist for Form1 and set the *IsMdiContainer* = *true*
 2. Add a menu strip onto Form1(Parent Form) and include the following menu options



3. Add another 3 forms to your application named Form2, Form3 and Form 4
4. When the user clicks a menu option to open a form, a new form should appear – the **form.show** method will ensure this. However, with this strategy you can have a situation where multiple windows/forms open up concurrently. This is not good, but it gets the job done!!
5. The proper strategy to handle the inefficiency of the previous strategy is by incorporating the following logic:

Each time a menu option is selected:

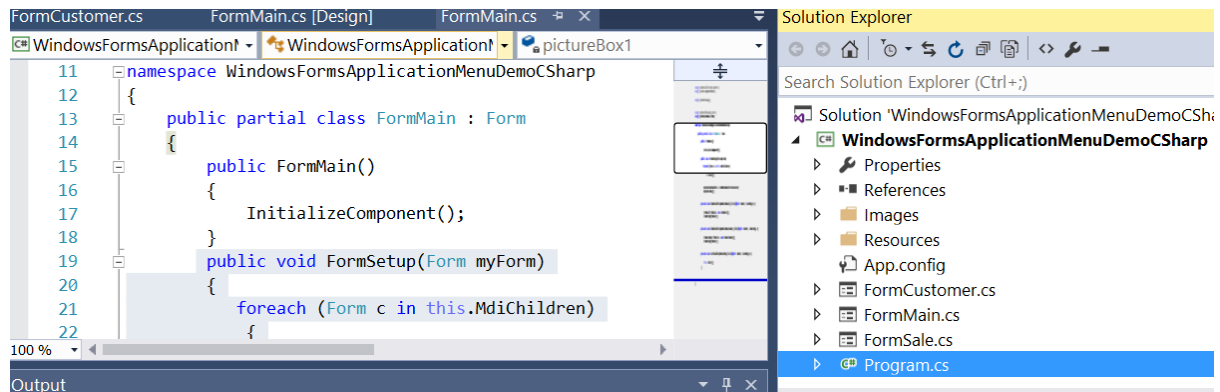
Check if any children are alive –kill them!

Set yourself to be parent of new child

Show new child

End method

The C# Code



```
public void FormSetup(Form myForm)
{
    //if a childform exists, close it
    if (this.ActiveMdiChild != null)
    {
        this.ActiveMdiChild.Close();
    }
    //create and load a childform
    ChildForm childForm = new ChildForm();
    //set the parent form of the child window
    childForm.MdiParent = this;
    //display the child window
    //maximize the child window
    childForm.WindowState = FormWindowState.Maximized;
    childForm.Show();
}

private void form1ToolStripMenuItemSale_Click(object sender, EventArgs e)
{
    FormSale frmSale = new FormSale();
    FormSetup(frmSale);
}

private void form2ToolStripMenuItemCustomer_Click(object sender, EventArgs e)
{
    FormCustomer frmCust = new FormCustomer();
    FormSetup(frmCust);
}

private void exitToolStripMenuItem_Click(object sender, EventArgs e)
{
    this.Close();
}

}
```

6. Call this sub-routine as the event handler for each menu option – i.e. double click the menu option and call the sub-routine by passing in the correct form name

Example of event handler for Form2 which could be used to handle a Sale – so let's call it FormSale:

```
private void sale1ToolStripMenuItem_Click(object sender, EventArgs e)
{
    FormSale1 frmSale1 = new FormSale1();
    PrepareForm(frmSale1);
}
```

Prac 1 Part Two: Major Project Menu Prototype (MDI)

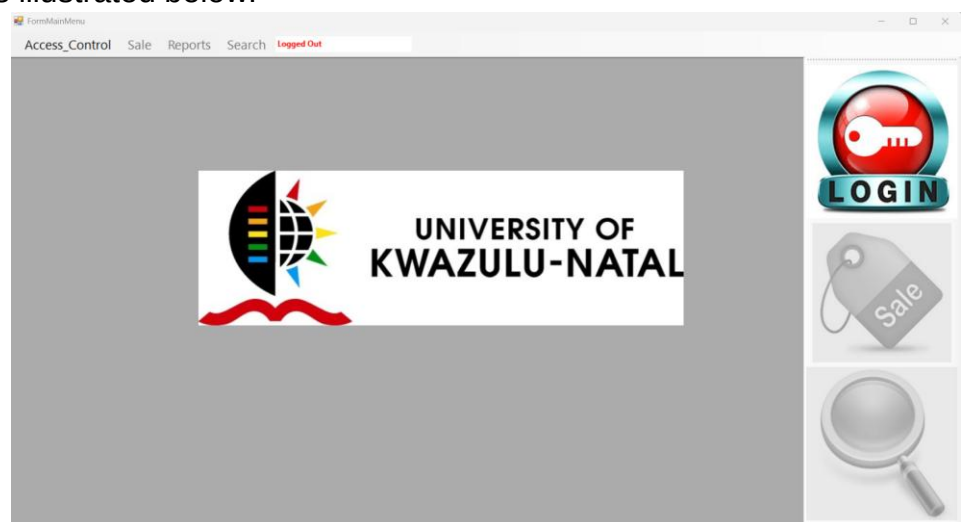
This is an opportunity for you to “*unleash*” your “creative juices” and develop a possible menu design for your main Major Project system. The menu design should include a few menu options (that can be arbitrarily named). Each menu option should have a few sub-menu options just for demonstration purpose. The actual content that is displayed when a menu option is selected could simply be a blank screens, with maybe just a single label showing that the menu has changed.

From a usability perspective, it is always good to make use of icons as a trigger to activate a menu option. This enhances the look and feel of the user interface.

Also, the access to the menu could be controlled by a log-in screen and only if the correct username and password has been entered, then the other menu options become available.

A demo of the processing is illustrated below:

The opening Screen...



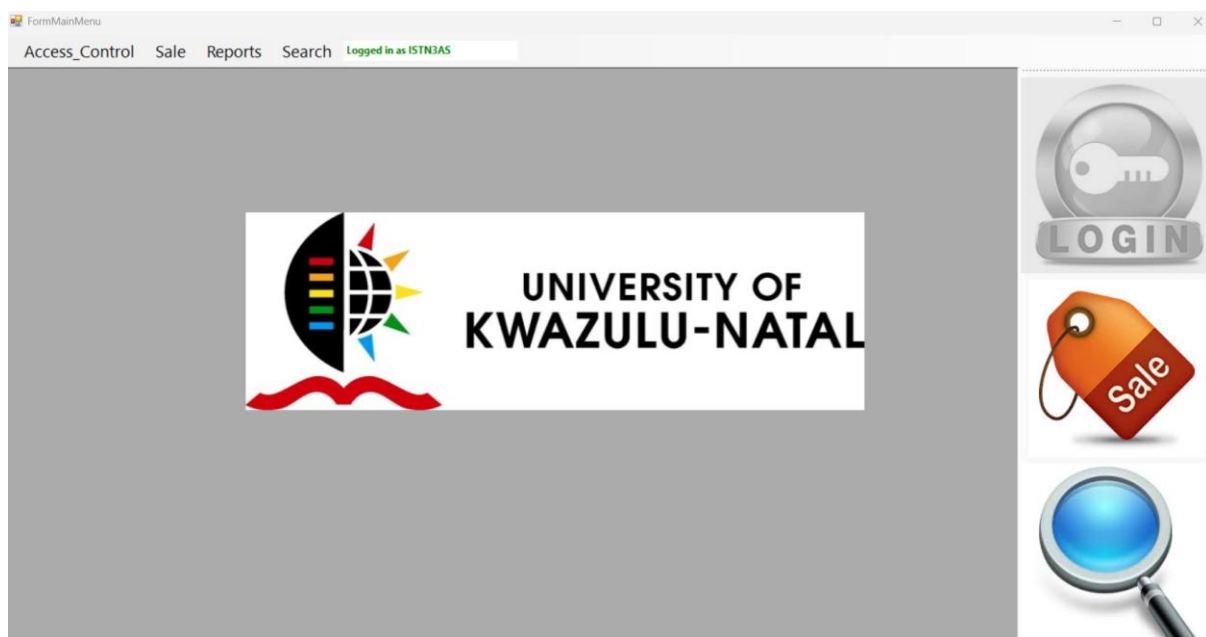
Notice – the Access control options ...some menu options are greyed out and only become available when the user logs-in.

For the time being, the security of the system will simply be simulated by attaching a username and password as “dummy” values so that the system opens up and allows access. The image icons can be greyed out by making use of the *enabled* property of the button onto which each image is attached. The dummy security is achieved via the following code for the event handler of the OK button from the Login form

```

if (txtUserName.Text=="ISTN3AS" && txtPassWord.Text=="2026")
{
    MessageBox.Show("WELCOME to the MAIN SYSTEM!! ");
    Form frm = (Form)Application.OpenForms["FormMainMenu"];
    MenuStrip ms = (MenuStrip)frm.Controls["menuStrip1"];
    ms.Items["saleMenu"].Enabled = true;
    ms.Items["reportMenu"].Enabled = true;
    ms.Items["searchMenu"].Enabled = true;
    ToolStripMenuItem ti = (ToolStripMenuItem)ms.Items["Access"];
    ti.DropDownItems["logoutMenu"].Enabled = true;
    ti.DropDownItems["loginMenu"].Enabled = false;
    ToolStrip ts = (ToolStrip)frm.Controls["toolStrip1"];
    ts.Items["toolStripLogin"].Enabled = false;
    ts.Items["toolStripSale"].Enabled = true;
    ts.Items["toolStripSearch"].Enabled = true;
    ToolStripTextBox tuser = (ToolStripTextBox)ms.Items["UserNameTxt"];
    tuser.ForeColor = Color.Green;
    tuser.Text = "Logged in as " + txtUserName.Text;
    this.Close();
}
}
}

```



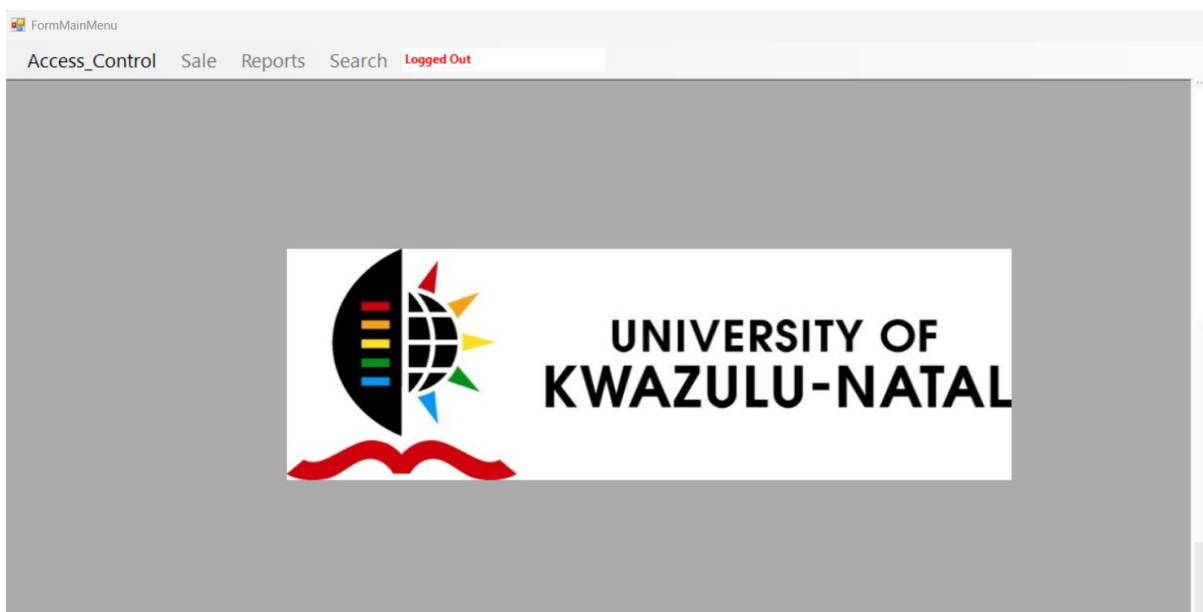
The processing logic entails setting a predefined username and password and then *enabling* all then menu items on the menustrip and the buttons on the toolstrip. This opens up the system for processing access...

...and once you have access...you can then navigate to any menu option...e.g. Report Form One or Two



...also, the icons on the side enable you to access the first form from each sub-menu... and also provide a primitive search facility.

...finally, the log-out...which takes you back to the log-in screen...



...and the exit which basically exits the application!